

An Improved Algorithm for Sorting by Block-Interchanges Based on Permutation Groups

Yen-Lin Huang¹, Cheng-Chen Huang², Chuan Yi Tang¹ and Chin Lung Lu^{2*}

¹Department of Computer Science, National Tsing Hua University, Hsinchu 300, Taiwan

²Institute of Bioinformatics Department of Biological Science
and Technology, National Chiao Tung University, Hsinchu 300, Taiwan
cllu@mail.nctu.edu.tw

Abstract

Given a chromosome represented by a permutation of genes, a block-interchange is proposed as a generalized transposition that affects a chromosome by swapping two nonintersecting segments of genes. The problem of sorting by block-interchanges is to find a minimum series of block-interchanges for transforming one chromosome into another. In this paper, we present an $\mathcal{O}(n + \delta \log n)$ time algorithm for solving the problem of sorting by block-interchanges, which improves a previous algorithm of $\mathcal{O}(\delta n)$ time proposed by Lin et al. [12], where n is the number of genes and δ is the minimum number of block-interchanges required for the transformation.

1 Introduction

Genome rearrangement studies based on genome-wide analysis of gene orders play an important role in the phylogenetic tree reconstruction [1–5]. To evaluate the evolutionary distance between two related genomes in gene order, various rearrangement events acting on genes within or among chromosomes have been proposed, such as reversals [3, 6–8], transpositions [9, 10] and block-interchanges [11–13], where *reversals*, also called *inversions*, affect a block of consecutive integers in the chromosome by reversing the order and flipping the signs of the integers; *transpositions* affect two adjacent blocks in the chromosome by exchanging their positions; *block-interchanges* are *generalized transpositions* by allowing the exchanged blocks not being adjacent in the chromo-

some. In genomes with multiple chromosomes, the rearrangement operations include translocations [2, 14–16], fusions [2, 17, 18] and fissions [2, 17, 18], where *translocations* exchange the end segments between two chromosomes; *fusions* join two chromosomes into a bigger one; *fissions* break a chromosome into two smaller ones.

Recently, the study on the genome rearrangement using block-interchanges has increasingly drawn great attention, since the block-interchange event is a generalization of transposition and, currently, its computational models measuring the genetic distance are more tractable than those modeled by transposition. Christie [11] first introduced the concept of block-interchange and also proposed an $\mathcal{O}(n^2)$ time algorithm using the breakpoint graph approach, where n is the number of genes, to solve so-called the problem of sorting by block-interchanges that is to find a minimum series of block-interchanges for transforming one chromosome into another. However, the chromosomes he considered were restricted to linear chromosomes. Later, by making use of permutation groups in algebra, Lin et al. [12] designed a simpler algorithm for solving the same problem on both linear or circular chromosomes with time-complexity of $\mathcal{O}(\delta n)$, where δ is the minimum number of block-interchanges required for the transformation (hence it is also called as a block-interchange distance problem) and can be calculated in $\mathcal{O}(n)$ time in advance. Recently, Feng and Zhu [13] proposed a new data structure called permutation tree to improve the Christie's algorithm by reducing the time complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$. In this paper, we derive a new property that help us to improve Lin et al.'s algorithm based on permutation groups by using the data structure of permutation tree, such that the overall time complexity can be further down to $\mathcal{O}(n + \delta \log n)$.

*Corresponding author: Dr. C.L. Lu, Department of Biological Science and Technology, National Chiao Tung University, Hsinchu 300, Taiwan. Email: cllu@mail.nctu.edu.tw.

The rest of this paper is organized as follows. Some basic concepts and properties of the permutation groups in algebra are introduced in Section 2. In Section 3, we introduce the idea of algorithm proposed by Lin *et al.* and present an improved algorithm of $\mathcal{O}(n + \delta \log n)$. Finally, we make a conclusion in Section 4.

2 Preliminaries

In group theory, a *permutation* is defined to be a one-to-one mapping from a set $E = \{1, 2, \dots, n\}$ into itself. For example, we may define a permutation α of the set $\{1, 2, 3, 4, 5\}$ by specifying $\alpha(1) = 4, \alpha(2) = 1, \alpha(3) = 2, \alpha(4) = 5$ and $\alpha(5) = 3$. In the study of genome rearrangement, it is convenient to express the permutation in *cycle* form as $\alpha = (1, 4, 5, 3, 2)$, in which for each $x \in E, \alpha(x)$ is placed directly right to x . A cycle of length k , say $(a_1, a_2, \dots, a_k)$, is simply called *k-cycle* and it also can be rewritten as $(a_i, a_{i+1}, \dots, a_k, a_1, \dots, a_{i-1})$ (i.e., indices are cyclic), where $2 \leq i \leq k$. Any two cycles are said to be *disjoint* if they have no element in common. In fact, any permutation, say α , can be written in a unique way as the product of disjoint cycles, which is called the *cycle decomposition* of α , if we ignore the order of the cycles in the product. Usually, a cycle of length one in α is not explicitly written and its element, say x , is said to be *fixed* by α since $\alpha(x) = x$. Especially, the permutation whose elements are all fixed is called an *identity permutation* and is denoted by $\mathbf{1}$, i.e., $\mathbf{1} = (1)(2) \cdots (n)$.

Given two permutations α and β of E , the *composition* (or *product*) of α and β , denoted by $\alpha\beta$, is defined to be a permutation of E with $\alpha\beta(x) = \alpha(\beta(x))$ for all $x \in E$. For instance, if we let $E = \{1, 2, 3, 4, 5, 6\}$, $\alpha = (2, 3)$ and $\beta = (2, 1, 5, 3, 6, 4)$, then $\alpha\beta = (2, 1, 5)(3, 6, 4)$. If α and β are disjoint cycles, then $\alpha\beta = \beta\alpha$. The *inverse* of α is defined to be a permutation, denoted by α^{-1} , such that $\alpha\alpha^{-1} = \alpha^{-1}\alpha = \mathbf{1}$. If a permutation is expressed by the product of disjoint cycles, then its inverse can be obtained by just reversing the order of the elements in each cycle. For example, if $\alpha = (2, 1, 5)(3, 6, 4)$, then $\alpha^{-1} = (5, 1, 2)(4, 6, 3)$. Clearly, $\alpha^{-1} = \alpha$ if α is a 2-cycle. Furthermore, it is well known that every permutation can be written as a product of 2-cycles. For example, $(1, 2, 3, 4) = (1, 4)(1, 3)(1, 2)$. However, there are many ways of expressing a permutation as a product of 2-cycles [20]. Given a permutation α , let $f(\alpha)$ denote the number of the

disjoint cycles in the cycle decomposition of α . Notice that $f(\alpha)$ counts also the non-expressed cycles of length one. For example, if $\alpha = (1, 5)(2, 4)$ is a permutation of $E = \{1, 2, \dots, 5\}$, then $f(\alpha) = 3$, instead of $f(\alpha) = 2$, since $\alpha = (1, 5)(2, 4)(3)$.

3 Sorting a permutation by block-interchanges

Recall that the problem of sorting by block-interchanges is to find a minimum series of block-interchanges for transforming a source chromosome into a target chromosome. By convention, the target chromosome is usually denoted by $I = (1, 2, \dots, n)$. Meidanis and Dias [17, 19] first noted that each cycle of a permutation may represent a circular chromosome of a genome with each element of the cycle corresponding to a gene, and the order of the cycle corresponding to the gene order of the chromosome. Figure 1, for example, shows a chromosome, which represented by $(1, 4, 5, 3, 2)$. Moreover, they observed that global evolutionary events, such as fusions and fissions (respectively, transpositions), correspond to the composition of a 2-cycle (respectively, 3-cycles) and the permutation representing a chromosome. For instance, let α be any permutation whose cycle decomposition is $\alpha_1\alpha_2 \cdots \alpha_r$. If $\rho = (x, y)$ is a 2-cycle and x and y are in the different cycles of α , say $\alpha_p = (a_1 \equiv x, a_2, \dots, a_i)$ and $\alpha_q = (b_1 \equiv y, b_2, \dots, b_j)$ where $1 \leq p, q \leq r$, then in the composition $\rho\alpha$, α_p and α_q are joined into a cycle $(a_1, a_2, \dots, a_i, b_1, b_2, \dots, b_j)$, i.e., ρ is a fusion event affecting on α (and also called a *join operation* of α here). If $\rho = (x, y)$ is a 2-cycle and x and y are in the same cycle of α , say $\alpha_p = (a_1 \equiv x, a_2, \dots, a_i \equiv y, a_{i+1}, \dots, a_j)$ where $1 \leq p \leq r$, then in the composition $\rho\alpha$, this cycle α_p is broken into two disjoint cycles $(a_1, a_2, \dots, a_{i-1})$ and $(a_i, a_{i+1}, \dots, a_j)$, i.e., ρ is a fission event affecting on α (and also called a *split operation* of α here). If $\rho = (x, y, z)$ is a 3-cycle and x, y and z are in the same cycle of α , say $\alpha_p = (a_1 \equiv x, a_2, \dots, a_i, b_1 \equiv y, b_2, \dots, b_j, c_1 \equiv z, c_2, \dots, c_k)$ where $1 \leq p \leq r$, then in the composition $\rho\alpha$, the cycle α_p becomes $(a_1, a_2, \dots, a_i, c_1, c_2, \dots, c_k, b_1, b_2, \dots, b_j)$, i.e., ρ is a transposition event affecting on α .

Recently, Lin *et al.* [12] further observed that a block-interchange affecting on α corresponds to two 2-cycles, say ρ_1 and ρ_2 , such that ρ_1 is a split operation of α and ρ_2 is a join operation of $\rho_1\alpha$. More clearly, let $\alpha_p = (a_1, a_2, \dots, a_k)$ be a cycle of α , $\rho_1 = (a_1, a_i)$ and $\rho_2 = (a_h, a_j)$, where

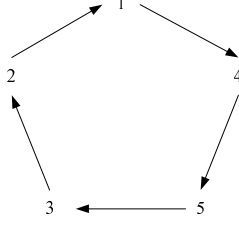


Figure 1: The illustration of a permutation $\alpha = (1, 4, 5, 3, 2)$ meaning that $\alpha(1) = 4, \alpha(2) = 1, \alpha(3) = 2, \alpha(4) = 5$ and $\alpha(5) = 3$.

$1 < i \leq k, 1 \leq h \leq i-1$ and $i \leq j \leq k$. Then $\rho_2 \rho_1 \alpha$ is the permutation obtained from α by exchanging the blocks $[a_h, a_{i-1}]$ and $[a_j, a_k]$ of α_p . That is, $\rho_2 \rho_1$ is a block-interchange event that affects on α by swapping $[a_h, a_{i-1}]$ and $[a_j, a_k]$, two non-intersecting blocks in α .

As discussed above, any series of block-interchanges required to transform one chromosome α into another I can be expressed by a product of 2-cycles, say $\rho_k \rho_{k-1} \dots \rho_1$, such that $\rho_k \rho_{k-1} \dots \rho_1 \alpha = I$ (hence, $\rho_k \rho_{k-1} \dots \rho_1 = I \alpha^{-1}$). This property implies that $I \alpha^{-1}$ contains all information that can be utilized to derive $\rho_1, \rho_2, \dots, \rho_k$ for transforming α into I .

Based on the above idea, Lin *et al.* [12] have used permutation groups to design an $\mathcal{O}(\delta n)$ time algorithm to sort a permutation by block-interchanges, where δ is the minimum number of block-interchanges required for the transformation. In [12], Lin *et al.* first showed that $\delta = \frac{n-f(I \alpha^{-1})}{2}$, which can be calculated in $\mathcal{O}(n)$ time in advance. They then used a greedy method to derive an optimal series of δ block-interchanges from $I \alpha^{-1}$. Recall that a block-interchange can be modeled by a join operation following a split operation. Indeed, in each iteration of the greedy method, they showed that the block-interchange can be obtained by first arbitrarily choosing two adjacent elements from $I \alpha^{-1}$ that serve as a split operation of α and then finding the corresponding join operation in $\mathcal{O}(n)$ time, where two elements x and y are said to be adjacent in a permutation β if $\beta(x) = y$ or $\beta(y) = x$. The details can be referred to Algorithm SortBI in this paper. It is not hard to see, the bottleneck of computation in the above algorithm is to find the join operations. Here, we observe the following property that can help us find the join operations in a more efficient way, so that the overall time complexity of Algorithm SortBI can be further improved.

Lemma 1. *Let α' be a permutation of $E =$*

$\{1, 2, \dots, n\}$ and $z \in E$. Suppose that z and $I(z)$ are in different cycles of α' . Then $\alpha'(z)$ and $I(z)$ are adjacent in $I \alpha'^{-1}$, and $(\alpha'(z), I(z))$ is a join operation of α' .

Proof. $I \alpha'^{-1}(\alpha'(z)) = I(z)$, and hence $\alpha'(z)$ and $I(z)$ are adjacent in $I \alpha'^{-1}$. Moreover, since z and $\alpha'(z)$ are in the same cycle of α' , $\alpha'(z)$ and $I(z)$ are in different cycles of α' . Therefore, $(\alpha'(z), I(z))$ is a join operation of α' . $\square$

Lemma 2. *Let α' be a permutation of $E = \{1, 2, \dots, n\}$, C be a cycle of α' , and z be the maximum element in C . Suppose that $z \neq n$. Then $\alpha'(z)$ and $I(z)$ are adjacent in $I \alpha'^{-1}$, and $(\alpha'(z), I(z))$ is a join operation of α' .*

Proof. Since z is in C and $I(z) = z + 1$, $I(z)$ is not in C . That is, z and $I(z)$ are in different cycles of α' . Clearly, this lemma holds according to Lemma 1. $\square$

By Lemma 2, we have the following observation immediately.

Observation 1. *Let α' be a permutation of E with two cycles C_1 and C_2 . Let m_1 and m_2 be the maximum elements in C_1 and C_2 , respectively. Suppose that $m_1 < m_2$, then $\alpha(m_1)$ and $I(m_1)$ are adjacent in $I \alpha'^{-1}$ and $(\alpha(m_1), I(m_1))$ is a join operation of α' .*

Based on the above observation, step 2.4 of Algorithm SortBI can be done through finding the maximum elements of the two cycles in the permutation. The details are described in Algorithm FastSortBI. The maximum element in a cycle of n different elements can be found in $\mathcal{O}(n)$ time. Actually, we observe that this job can be done more efficiently by using a data structure called permutation tree, which was first proposed by Feng and Zhu [13].

Given a permutation $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, a *permutation tree* of π is defined to be a balanced binary tree with n leaves, whose nodes are labelled according to the following two rules. (1) the leaves are labelled by $\pi_1, \pi_2, \dots, \pi_n$, respectively, in this order. (2) The other nodes are labelled in a way such that labels of parent nodes are greater than those of their children. By definition, the root r of a permutation tree has a label that is the maximum element in π . Basically, there are following three operations that can be applied to the permutation trees. They are (1) *Build*, which is to build the permutation tree of a permutation, (2) *Join*, which is to join two permutation trees into

Algorithm SortBI**Input:** A permutation $\alpha = (a_1, a_2, \dots, a_n)$;**Output:** A minimum series of block-interchange operations $\sigma_1, \sigma_2, \dots, \sigma_\delta$ for transforming α into I .

```

1:   Let  $\delta = \frac{n-f(I\alpha^{-1})}{2}$ ;
2:   for  $i = 1$  to  $\delta$  do
2.1:   Arbitrarily two adjacent elements  $x$  and  $y$  in  $I\alpha^{-1}$ ;
      /* Note that  $(x, y)$  is a split operation of  $\alpha$ . */
2.2:   Circularly shift  $(a_1, a_2, \dots, a_n)$  such that  $a_1 = x$  and assume  $y = a_k$ ;
2.3:   for  $j = 1$  to  $n$  do
      index( $a_j$ ) =  $j$ ;
      end for
2.4:   Find two adjacent elements  $u$  and  $v$  in  $I\alpha^{-1}(x, y)$  such that
      index( $u$ )  $\leq k - 1$  and index( $v$ )  $\geq k$ ;
      /* Note that  $(u, v)$  is a join operation of  $(x, y)\alpha$ . */
2.5:    $\sigma_i = (u, v)(x, y)$ ;
2.6:   Compute  $(u, v)(x, y)\alpha$  and denote it by  $\alpha$  again;
2.7:   Compute  $I\alpha^{-1}(x, y)(u, v)$  and denote it by  $I\alpha^{-1}$  again;
      end for
3:   Output  $\sigma_1, \sigma_2, \dots, \sigma_\delta$ ;

```

one, and (3) *Split*, which is to split one permutation tree into two. Feng and Zhu [13] showed that the costs of the *Build*, *Join* and *Split* operations are $\mathcal{O}(n)$, $\mathcal{O}(\log n)$ and $\mathcal{O}(\log n)$, respectively.

Theorem 1. *The block-interchange distance problem can be solved by Algorithm FastSortBI in $\mathcal{O}(n + \delta \log n)$ time.*

Proof. As discussed previously, Algorithm FastSortBI transforms α into I using a minimum number of block-interchange operations. Now, we analyze its time-complexity as follows. It is clear that the computation of step 1 can be done in $\mathcal{O}(n)$ time. As to step 2, there are δ iterations and in each iteration step 2.3 of finding the maximum elements in the cycles is the computational bottleneck in Algorithm FastSortBI. As discussed above, it can be implemented in $\mathcal{O}(\log n)$ time by using the permutation trees, whose roots keep the maximum values of all these cycles in the permutation. The details are described as follows. Initially, we build a permutation tree of the input permutation α . Then, finding a split operation for α can be done either by applying a *Split* operation, or two *Split* and one *Join* operations to its corresponding permutation tree, which costs $\mathcal{O}(\log n)$ time. After splitting α into two cycles, we find the maximum elements of the two cycles from roots of their corresponding permutation trees in constant time. Next, finding a join operation for the above two cycles can be done either by applying a *Join* operation, or three *Join* and two *Split* operations to the

permutation trees, which still costs $\mathcal{O}(\log n)$ time. As a result, step 2 takes $\mathcal{O}(\log n)$ time in each of δ iterations. Therefore, the total time-complexity of Algorithm FastSortBI is $\mathcal{O}(n + \delta \log n)$. $\square$

Let us take $\alpha = (7, 1, 4, 6, 2, 5, 3, 8)$ for an example. Initially, we build a permutation tree T from α . We note that $I\alpha^{-1} = (1, 8, 4, 2, 7)(3, 6, 5)$ and $n = 8$. By Algorithm FastSortBI, we understand that the block-interchange distance between α and I is $\delta = \frac{(8-2)}{2} = 3$, which means that α can be transformed into I using three block-interchange operations. Next, we show how to find these three block-interchange operations, say $\sigma_1 = \rho_{12}\rho_{11}$, $\sigma_2 = \rho_{22}\rho_{21}$, and $\sigma_3 = \rho_{32}\rho_{31}$. We let $\rho_{11} = (1, 8)$ since 1 and 8 are adjacent in $I\alpha^{-1}$. After applying ρ_{11} to α , we obtain $\alpha' = \rho_{11}\alpha = (1, 8)(7, 1, 4, 6, 2, 5, 3, 8) = (1, 4, 6, 2, 5, 3)(8, 7)$. Actually, the result of this split operation on α can be implemented by applying two *Split* operations and one *Join* operations to the permutation tree T of α . The two *Split* divide T into three subtrees T_1 , T_2 and T_3 , where T_1 , T_2 and T_3 correspond to (7) , $(1, 4, 6, 2, 5, 3)$ and (8) , respectively. Then the *Join* merge T_3 and T_1 into a new tree denoted by T_{31} , corresponding to $(8, 7)$. We note here that T_2 and T_{31} correspond to the permutation α' that is equal to $\rho_{11}\alpha$. Next, in step 2.3, we find $z = 6$ and hence $\rho_{12} = (\alpha'(6), I(6)) = (2, 7)$ is a join operation of α' , since the maximum values of T_2 and T_{31} are 6 and 8, respectively. Similarly to

Algorithm FastSortBI**Input:** A permutation $\alpha = (a_1, a_2, \dots, a_n)$;**Output:** A minimum series of block-interchange operations $\sigma_1, \sigma_2, \dots, \sigma_\delta$ for transforming α into I .

```

1:  Let  $\delta = \frac{n-f(I\alpha^{-1})}{2}$ ;
2:  for  $i = 1$  to  $\delta$  do
2.1:  Arbitrarily choose two adjacent elements  $x$  and  $y$  in  $I\alpha^{-1}$ ;
      /* Note that  $(x, y)$  is a split operation of  $\alpha$ . */
2.2:   $\alpha' = (x, y)\alpha$ ;
      /* Note that  $\alpha'$  has two cycles, say  $C_1$  and  $C_2$ . */
2.3:  Find the maximum element  $m_1$  in  $C_1$  and  $m_2$  in  $C_2$ ;
      Let  $z = \min(m_1, m_2)$ ;
      Let  $u = \alpha'(z)$  and  $v = I(z)$ ;
      /* Note that  $(u, v)$  are adjacent in  $I\alpha'^{-1}$ , where  $(u, v)$  is a join operation of  $\alpha'$ . */
2.4:   $\sigma_i = (u, v)(x, y)$ ;
2.5:  Compute  $(u, v)\alpha'$  and denote it by  $\alpha$  again;
2.6:  Compute  $I\alpha^{-1}(x, y)(u, v)$  and denote it by  $I\alpha^{-1}$  again;
    end for
3:  Output  $\sigma_1, \sigma_2, \dots, \sigma_\delta$ ;

```

the above split operation, this join can be done by applying two *Split* operations and three *Join* operations to T_2 and T_{31} . The two *Split* operations first divide T_2 and T_{31} into four subtrees, and then the three *Join* operations merge them into a new tree that corresponds to $(2, 5, 3, 1, 4, 6, 7, 8)$. In the second iteration, we have $\rho_{21} = (1, 4)$, $\alpha' = (1)(4, 6, 7, 8, 2, 5, 3)$, $\rho_{22} = (\alpha'(1), I(1)) = (1, 2)$ and $\alpha = (1, 2, 5, 3, 4, 6, 7, 8)$. Finally, we let $\rho_{31} = (3, 6)$ and $\rho_{32} = (3, 5)$ and, therefore, $I\alpha^{-1}\alpha = (3, 5)(3, 6)(1, 2, 5, 3, 4, 6, 7, 8) = (3, 4, 5, 6, 7, 8, 1, 2) = I$.

4 Conclusions

In this paper, we studied the problem of sorting by block-interchanges that is to find a minimum series of block-interchanges to sort the permutation of the given chromosome. Here, we proposed an improved algorithm of $\mathcal{O}(n + \delta n)$ time based on permutation groups.

References

- [1] D. Sankoff, G. Leduc, N. Antoine, B. Paquin, B. F. Lang, and R. Cedergren, "Gene order comparisons for phylogenetic inference: evolution of the mitochondrial genome," *Proceedings of the National Academy of Sciences*, vol. 89, pp. 6575–6579, 1992.
- [2] S. Hannenhalli and P. A. Pevzner, "Transforming men into mice (polynomial algorithm for genomic distance problem)," in *Proceedings of the 36th IEEE Symposium on Foundations of Computer Science (FOCS 1995)*. IEEE Computer Society, 1995, pp. 581–592.
- [3] S. Hannenhalli and P. A. Pevzner, "Transforming cabbage into turnip: Polynomial algorithm for sorting signed permutations by reversals," *Journal of the ACM*, vol. 46, pp. 1–27, 1999.
- [4] P. Pevzner and G. Tesler, "Genome rearrangements in mammalian evolution: lessons from human and mouse genomes," *Genome Research*, vol. 13, pp. 37–45, 2003.
- [5] E. Belda, A. Moya, and F. J. Silva, "Genome rearrangement distances and gene order phylogeny in γ -Proteobacteria," *Molecular Biology Evolutionary*, vol. 22, pp. 1456–1467, 2005.
- [6] H. Kaplan, R. Shamir, and R. E. Tarjan, "Faster and simpler algorithm for sorting signed permutations by reversals," *SIAM Journal on Computing*, vol. 29, pp. 880–892, 1999.
- [7] D. A. Bader, B. M. Moret, and M. Yan, "A linear-time algorithm for computing inversion distance between signed permutations with an experimental study," *Journal of Computational Biology*, vol. 8, pp. 483–491, 2001.

- [8] E. Tannier, A. Bergeron, and M.-F. Sagot, "Advances on sorting by reversals," *Discrete Applied Mathematics*, vol. 155, pp. 881–888, 2007.
- [9] V. Bafna and P. A. Pevzner, "Sorting by transpositions," *SIAM Journal on Discrete Mathematics*, vol. 11, pp. 221–240, 1998.
- [10] I. Elias and T. Hartman, "A 1.375-approximation algorithm for sorting by transpositions," in *Proceedings of the 5th Workshop on Algorithms in Bioinformatics (WABI 2005)*, ser. Lecture Notes in Computer Science, R. Casadio and G. Myers, Eds., vol. 3692. Springer-Verlag, 2005, pp. 204–215.
- [11] D. A. Christie, "Sorting by block-interchanges," *Information Processing Letters*, vol. 60, pp. 165–169, 1996.
- [12] Y. C. Lin, C. L. Lu, H.-Y. Chang, and C. Y. Tang, "An efficient algorithm for sorting by block-interchanges and its application to the evolution of vibrio species," *Journal of Computational Biology*, vol. 12, pp. 102–112, 2005.
- [13] J. X. Feng and D. M. Zhu, "Faster Algorithms for Sorting by Transpositions and Sorting by Block Interchanges" *ACM Transactions on Algorithms*, vol. 3, No. 3, 2007.
- [14] S. Hannenhalli, "Polynomial algorithm for computing translocation distance between genomes," *Discrete Applied Mathematics*, vol. 71, pp. 137–151, 1996.
- [15] A. Bergeron, J. Mixtacki, and J. Stoye, "On sorting by translocations," *Journal of Computational Biology*, vol. 13, pp. 567–578, 2006.
- [16] M. Ozery-Flato and R. Shamir, "An $O(n^{3/2}\sqrt{\log n})$ algorithm for sorting by reciprocal translocations," in *Proceedings of the 17th Annual Symposium on Combinatorial Pattern Matching (CPM 2006)*, ser. Lecture Notes in Computer Science, M. Lewenstein and G. Valiente, Eds., vol. 4009. Springer, 2006, pp. 258–269.
- [17] J. Meidanis and Z. Dias, "Genome rearrangements distance by fusion, fission, and transposition is easy," in *Proceedings of the 8th International Symposium on String Processing and Information Retrieval (SPIRE 2001)*, G. Navarro, Ed. IEEE Computer Society, 2001, pp. 250–253.
- [18] C. L. Lu, Y. L. Huang, T. C. Wang, and H.-T. Chiu, "Analysis of circular genome rearrangement by fusions, fissions and block-interchanges," *BMC Bioinformatics*, vol. 7, no. 295, 2006.
- [19] J. Meidanis and Z. Dias, "An alternative algebraic formalism for genome rearrangements," in *Comparative Genomics: Empirical and Analytical Approaches to Gene Order Dynamics, Map Alignment and Evolution of Gene Families*, D. Sankoff and J. H. Nadeau, Eds. Kluwer Academic Press, 2000, pp. 213–223.
- [20] J. B. Fraleigh, *A First Course in Abstract Algebra*, 7th ed. Addison-Wesley, 2003.
- [21] E. R. Tillier and R. A. Collins, "Genome rearrangement by replication-directed translocation," *Nat Genet*, vol. 26, pp.195-197, 2000.